

EXPERIMENT 6

AIM: To implement mutual exclusion algorithm

Objective: Develop a program to implement mutual exclusion algorithm

THEORY:

Mutual exclusion : makes sure that concurrent process access shared resources or data in a serialized way. If a process , say P_i , is executing in its critical section, then no other processes can be executing in their critical sections

Token ring is one among mutual exclusion algorithm in distributed systems that does not involve with a server or coordinator.

Working

A token is passed from one node to another node in a logical order. A node received token has a proper right to enter the critical section. If a node being holding token does not require to enter critical section or access a shared resource, it just simply passes the token to the next node.

Advantages

- Process suffer from no starvation. Before entering a critical section, a process waits at most for the duration that all other processes enter and exit the critical section
- No hand shake messages are required to get the token. Then less overhead

Disadvantages

- If a token is lost for some reasons, another token must be regenerated
- Detecting lost token is difficult since there is no upper bound in the time a token takes to rotate the ring (no particular timeout)
- The ring must be reconstructed when a process crashes

Code:

```
import threading
import time
import random
```

```
class MaekawaProcess:
```

```
    def __init__(self, pid, quorum):
        self.pid = pid # Process ID
        self.quorum = quorum # Voting set
        self.lock = threading.Lock() # Ensures thread safety
        self.voted = None # Stores the process to which the vote is granted
        self.request_queue = [] # Queue of pending requests
```

```
    def request_cs(self):
        """Request Critical Section access."""
        print(f"Process {self.pid} requesting CS")
        self.request_queue.append(self.pid)
```

```
        # Send request to all quorum members
        for process in self.quorum:
            process.receive_request(self.pid)
```

```
        while self.request_queue[0] != self.pid:
            time.sleep(0.1) # Wait until it gets permission
```

```
        print(f"Process {self.pid} entering CS")
        time.sleep(random.uniform(1, 3)) # Simulate critical section execution
        print(f"Process {self.pid} exiting CS")
```

```

self.release_cs()

def receive_request(self, sender_pid):
    """Handle an incoming CS request."""
    with self.lock:
        if self.voted is None:
            self.voted = sender_pid
            print(f"Process {self.pid} grants vote to {sender_pid}")
        else:
            print(f"Process {self.pid} cannot vote for {sender_pid}, already voted for {self.voted}")

def release_cs(self):
    """Release the Critical Section and notify quorum members."""
    self.request_queue.pop(0)

    # Notify all quorum members
    for process in self.quorum:
        process.receive_release(self.pid)

def receive_release(self, sender_pid):
    """Handle CS release notification."""
    with self.lock:
        if self.voted == sender_pid:
            print(f"Process {self.pid} revokes vote from {sender_pid}")
            self.voted = None

# Get user input for number of processes
num_processes = int(input("Enter the number of processes: "))
processes = []

# Create process objects
for i in range(1, num_processes + 1):
    processes.append(MaekawaProcess(i, []))

# Assign quorum sets (Each process has a set of 2 or more quorum members)
for i in range(num_processes):
    processes[i].quorum = [processes[(i + 1) % num_processes], processes[(i + 2) % num_processes]]

# Create and start threads for each process
threads = []
for process in processes:
    t = threading.Thread(target=process.request_cs)
    threads.append(t)
    t.start()

# Wait for all threads to complete
for t in threads:
    t.join()

```

Output:

```

Enter the number of processes: 5
Process 1 requesting CS
Process 2 grants vote to 1
Process 3 grants vote to 1
Process 1 entering CS
Process 3 requesting CS
Process 2 requesting CS

```

Process 4 grants vote to 3
Process 5 grants vote to 3
Process 5 requesting CS
Process 4 requesting CS
Process 3 cannot vote for 2, already voted for 1
Process 1 grants vote to 5
Process 2 cannot vote for 5, already voted for 1
Process 3 entering CS
Process 4 cannot vote for 2, already voted for 3
Process 2 entering CS
Process 5 entering CS
Process 5 cannot vote for 4, already voted for 3
Process 1 cannot vote for 4, already voted for 5
Process 4 entering CS
Process 4 exiting CS
Process 1 exiting CS
Process 2 revokes vote from 1
Process 3 revokes vote from 1
Process 5 exiting CS
Process 1 revokes vote from 5
Process 3 exiting CS
Process 4 revokes vote from 3
Process 5 revokes vote from 3
Process 2 exiting CS

=== Code Execution Successful ===

Conclusion:

Maekawa's algorithm guarantees Mutual Exclusion (M.E.) by ensuring that each process gets permission from a quorum, and no two processes in overlapping quorums can enter the Critical Section (CS) simultaneously. The voting mechanism prevents multiple grants, and queued requests are served in timestamp order, ensuring fairness and preventing deadlocks.